

Fundamentals of Artificial Intelligence

Course Details

Title: Artificial Intelligence
Code: DAT6002 S2_25
Term: Spring 2026
Assessment: WRIT1

Student Details

Tom Rowan ST20285213

Project Tutor

Dr. Imtiaz Hussain Khan

Table of Contents

Part 1 - Symbolic AI	3
Task 1: Semantic Network Design for Medieval Weaponry	3
Structural Design and Abstraction	3
Inheritance	4
Task 2: Semantic Network Implementation	5
Python Implementation	5
Prolog Implementation	6
Sample Queries	8
Ethical Considerations	12
Part 2: Connectionist Methods	13
Dataset Preparation	13
A Biased Dataset	13
Outliers	14
Encoding and Scaling	18
One-Hot Encoding	18
Binary Variable Encoding	18
Normalisation Functions	18
Removal of Problematic Features	19
Correlation with Dependent Variable 'y'	20
Post Processing	21
Implementation of ANN From Scratch	22
Network Architecture	22
Feed Forward Pass	23
Back Propagation	23
Weight and Bias Update Step	24
Performance Evaluation	25
ANN Implementation with Libraries	29
Choice of Library	29
Model Parameter Configuration	29
Performance Comparison	30
Advantages and Disadvantages	31
Parameter Tuning and Model Optimisation	33
GridSearchCV	33
Genetic Algorithm Optimisation	35
References	40
Part 1 – Symbolic AI	40
Part 2 – Connectionist Methods	41

Part 1 - Symbolic AI

Task 1: Semantic Network Design for Medieval Weaponry

A semantic network is a way of representing knowledge about a subject by modelling relationships and attributes as 'nodes' and connecting 'edges' on a graph. In this project, the subject being represented is medieval weaponry, with the 'medieval period' defined as starting with the fall of the Western Roman Empire in 476 AD through to the beginning of the Protestant Renaissance Period in 1517 AD. (Study.com n.d., Metropolitan Museum 1991, Wikipedia 2026)

Structural Design and Abstraction

The semantic network was constructed using a hierarchical classification system, grouping items by shared properties and function. This follows the taxonomical classification found in biological classification used to group organisms by shared traits. (Campbell 2005).

An attribute is a possible general characteristic known to the network, while a property is an instance of an attribute that is attached to a distinct item.

In the design phase, choices were made about the abstraction to reflect frequently shared attributes. When several weapons shared properties (such as the combination of a blade, a hilt, and use at close quarters) it became more efficient to abstract these properties into a dedicated intermediate node, such as 'bladed_hand_weapon'. This approach avoids repeating identical 'has_a' facts.

Items that are closer to the root have fewer but more general properties. Conversely, objects that are leaves are likely to have more properties. Items inherit properties from objects that are closer to the root; these can be overridden (hidden) by more specific properties.

The design separates the taxonomy where an item belongs to a class from attributes belonging to the item.

A challenge emerged regarding whether to use multiple inheritance. While a tree structure works for most weapons, certain items are not simple to categorise. For example, trebuchet functions both as a siege weapon and a ranged weapon.

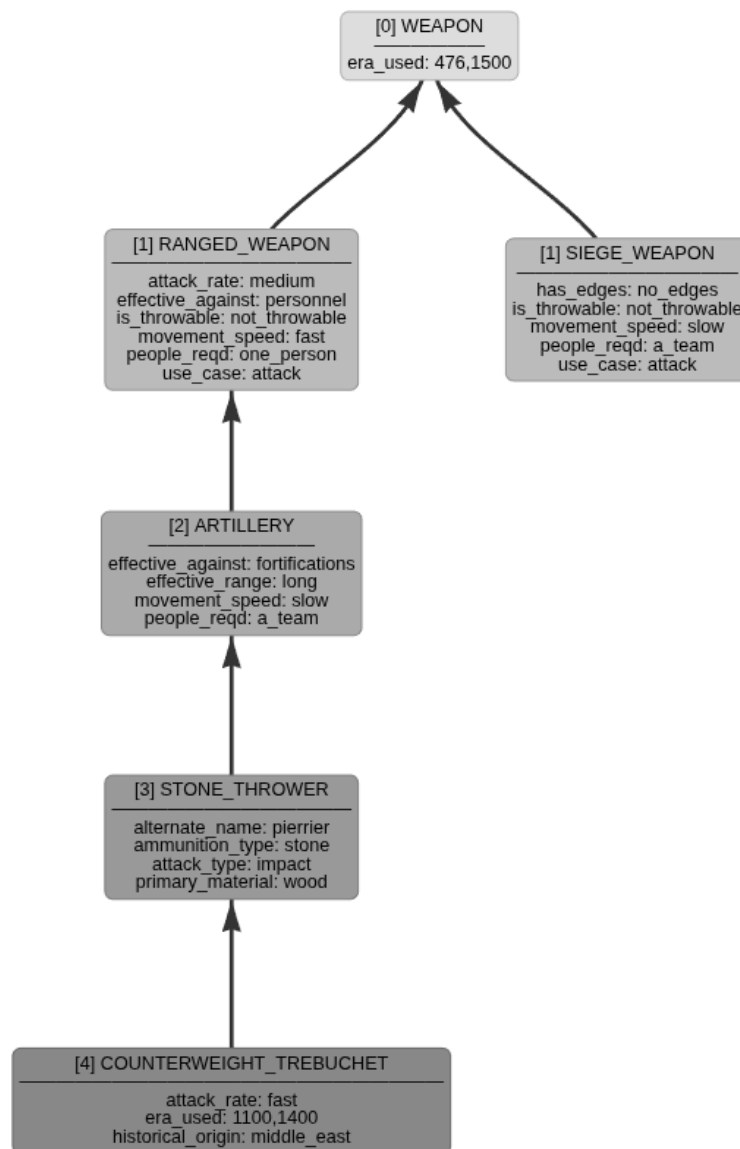


Figure 1 Showing the tree with the Trebuchet as a Ranged Weapon.

Inheritance

My design uses a strict tree without multiple inheritance. This is known as a mono-hierarchy. To fully capture the trebuchet, the tree would have to be modified to allow multiple inheritance. This highlights the difference between the simplicity of a tree and greater expressiveness allowed for by a poly-hierarchical multi-inheritance approach implemented as a Directed Acyclic Graph (DAG).

To ensure historical accuracy, the specific attributes and properties assigned to individual weapon nodes were taken from various resources, including John McDonald's 2015 website *'Medieval Weapons & Armour'*, Charles Boutell's book from 1893 *'Arms and Armour'* and the Metropolitan Museum of Art's 1991 *'survey of arms and armour'*. This research aims to provide a plausible foundation to the network.

Task 2: Semantic Network Implementation

The network was implemented in both Prolog and Python.

Python Implementation

A python class was created so a user can model any generic semantic network. It allows the user to add nodes and edges to the graph as 'is_a' and 'has_a' relationships. This approach represents knowledge as high-level human readable symbols. This allows for explicit knowledge representation. (Russell 2022).

From the resulting network, the user can output JSON, CSV and Prolog files. The class can also create interactive graphical visualisations which was the primary reason for implementing in python.

As the network is sizeable, containing 84 'is_a' relationships and 341 'has_a' properties, output visualisations are too detailed to show in full. They can be viewed here:

Visualisation Type	URL
Hierarchical Tree	https://datascience.daemonchild.com/year3/dat6002/Medieval_Weapons_Hierarchy.html
Network Node Diagram	https://datascience.daemonchild.com/year3/dat6002/Medieval_Weapons_Node_Diagram.html

The implementation creates network node and hierarchical tree diagrams, including filtered versions that include only selected nodes (as used in this report).

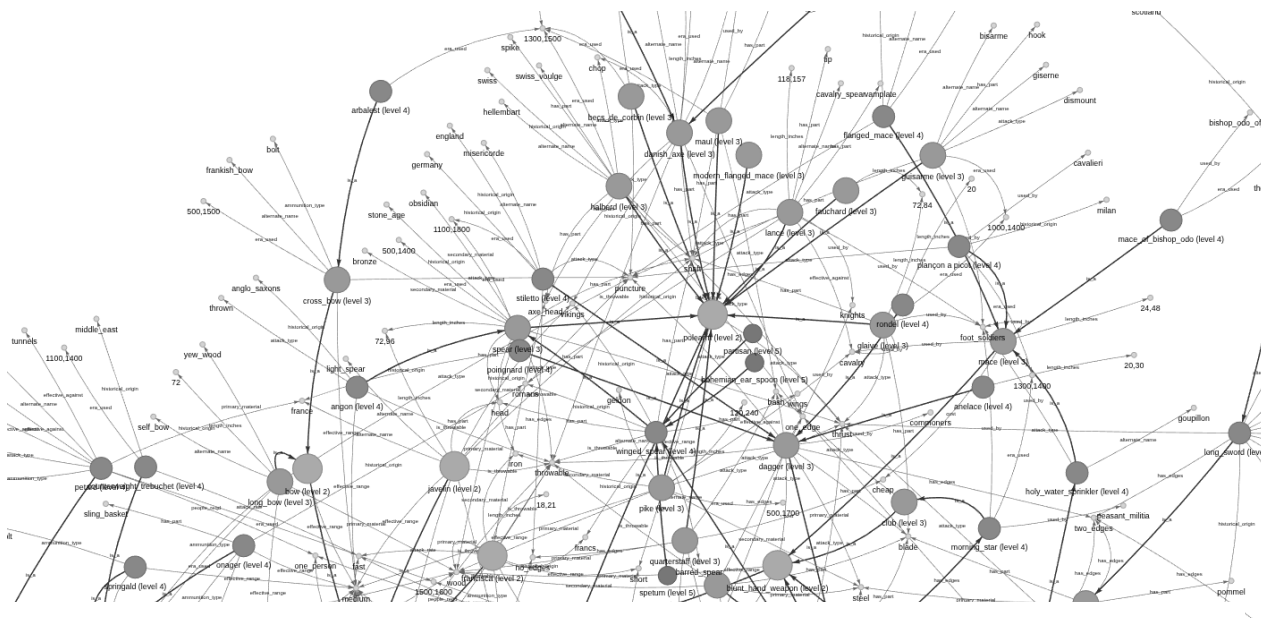


Figure 2 Showing a small part of a node-based visualisation.

There were several challenges in developing the python implementation. The NetworkX library is used to implement the underlying graph storage, storing items as nodes and relationships as labelled edges. It does not natively make a distinction between 'is_a' and 'has_a' edges. The 'get_node_level' function calculates the 'distance' from the root and avoids the network taking shortcuts via 'has_a' edges by only traversing 'is_a' paths.

Another challenge was configuring the model parameters of the PyVis library produce clear, useful visualisations.

Prolog Implementation

The python class does not provide an inference engine, so the network is exported to a Prolog file.

In Prolog the network is implemented using two types of fact:

- is_a facts
- has_a facts

The relationships between items in the network are defined in a parent-child hierarchy using the predicate 'is_a'. This builds a tree of relationships in the form is_a (child, parent).

```
is_a( melee_weapon, weapon ).  
is_a( bladed_hand_weapon, melee_weapon ).  
is_a( sword, bladed_hand_weapon ).
```

The properties of an object are defined as 'has_a' facts:

```
has_a( melee_weapon, effective_against, personnel ).  
has_a( bladed_hand_weapon, effective_range, close_quarters ).  
has_a( sword, has_part, blade ).  
has_a( sword, has_part, hilt ).  
has_a( sword, has_part, pommel ).
```

By defining nodes such as 'bladed_hand_weapon', the network uses shared inheritance. Each class below this, for example 'sword', automatically inherits properties like 'effective range'. Querying the network for the unique attributes defined for a generic sword would therefore return the following, with the attributes being inherited from further up the tree:

- has_part (from sword)
- effective_range (from blade_hand_weapon)
- effective_against (from melee_weapon)

The Closed World Assumption (CWA) is a fundamental tenet of Prolog. Any statement not provably true is assumed false. For example: `has_a (sword, colour, pink)` is not specifically stated, and cannot be inherited from a parent, so Prolog will conclude that this is false. Taking this further, the inference engine uses the Negation as Failure (NAF) principle. If the system attempts to prove a sword is pink and finds no supporting evidence the proof fails, and the statement is treated as logically false. (Clark 1978)

The CWA also ensures that the Prolog inference engine operates within a defined plausible version of reality. While other facts may be possible – such as a pink sword – the model is unable to hallucinate beyond the defined network. While it is physically possible for a medieval sword to have been forged from copper or decorated in pink, such facts are not plausible within this specific semantic network knowledgebase.

This means the model cannot infer anything outside of its knowledgebase.

In addition to the facts, rules have been written for querying this network. Most of these functions are generic and work for any semantic network.

- `is_a_member` - check class/parent membership
- `has_a_property` - does an item have a property?
- `list_all_items` - list all items in an attractive way
- `get_all_items` - get all items into a sorted list
- `list_all_attributes` - list all attributes in an attractive way
- `get_all_attributes` - get all attributes into a sorted list
- `list_properties` - list the defined properties for an item (including inherited)
- `list_items_with_property` - list all items with a property value
- `show_tree` – show all items in a tree

The following functions are specific to medieval weaponry:

- `longer_range` - compares weapon range
- `list_weapons_by_year` - find all weapons in use in a specific year

The Prolog inference engine uses deductive reasoning, where specific conclusions (that ‘the wallace sword’ is a melee weapon) are derived only from the supplied facts and rules. If the premises are true, the conclusion must also be true. (Russell 2022).

Inductive reasoning involves generating general rules based on specific observations. For example: “Several swords were made of steel, therefore all swords are made of steel.”

Abductive reasoning aims to find the most likely explanation for an incomplete set of observations. The system performs deductive reasoning only; it does not support inductive or abductive inference.

Sample Queries

The following queries were run under SWI Prolog version 9.0.4 on Linux.

It is always possible for Prolog to answer questions about the facts directly, for example: `is_a(sword, ranged_weapon)` which is false.

Using the rules network can also answer questions about parent relationships. The following examples ask the question “Is a long sword a melee weapon?”, and asks Prolog to show the lineage of a long sword:

```
? is_a_member(long_sword, melee_weapon).  
true .
```

```
?- is_a_member(long_sword, W).  
W = sword ;  
W = bladed_hand_weapon ;  
W = melee_weapon ;  
W = weapon ;
```

The same concept works for properties. This example shows the local properties of the sword class, firstly using the raw facts (no inheritance):

```
?- has_a(sword, Property, Value).  
Property = has_edges,  
Value = two_edges ;  
Property = has_part,  
Value = blade ;  
Property = has_part,  
Value = hilt ;  
Property = has_part,  
Value = pommel ;  
Property = primary_material,  
Value = steel.
```

Using the `has_a_property` rule to also obtains inherited properties from further up the tree:

```
?- has_a_property(sword, Property, Value).
Property = has_edges,
Value = two_edges ;
Property = has_part,
Value = blade ;
Property = has_part,
Value = hilt ;
Property = has_part,
Value = pommel ;
Property = primary_material,
Value = steel ;
Property = attack_type,
Value = cut ;
Property = effective_range,
Value = close_quarters ;
Property = ammunition_type,
Value = no_ammunition ;
Property = effective_against,
Value = personnel ;
Property = is_throwable,
Value = not_throwable ;
Property = movement_speed,
Value = fast ;
Property = people_reqd,
Value = one_person ;
Property = use_case,
Value = attack ;
Property = era_used,
Value = (476, 1500) ;
```

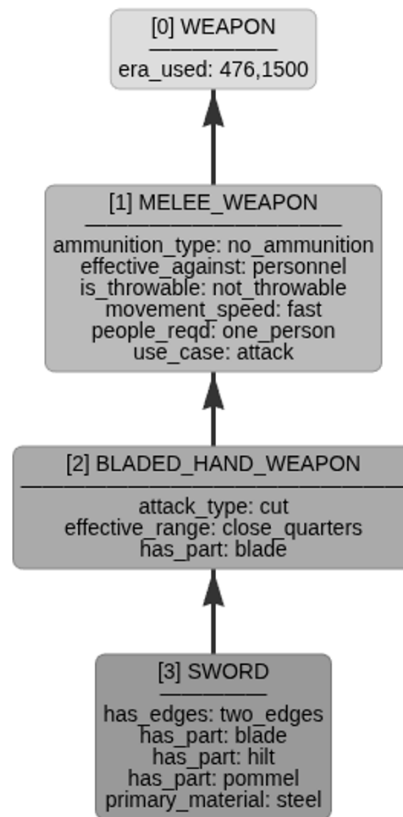


Figure 3 Showing properties in one branch of the tree.

My `list_properties` rule acts as a wrapper to make the same output easier to read:

```
?- list_properties(sword).
Properties for sword:
-----
- ammunition_type = no_ammunition
- attack_type = cut
- effective_against = personnel
- effective_range = close_quarters
- era_used = 476,1500
- has_edges = two_edges
- has_part = blade
- has_part = hilt
- has_part = pommel
- is_throwable = not_throwable
- movement_speed = fast
- people_reqd = one_person
- primary_material = steel
- use_case = attack
```

Sections of a 'trace' for the above query show how the inference engine searches the tree:

```
[trace] 29 ?- has_a_property(sword, Property, Value).
  Call: (12) has_a_property(sword, _41916, _41918) ? creep
  Call: (13) has_a(sword, _41916, _41918) ? creep
  Exit: (13) has_a(sword, has_edges, two_edges) ? creep
  Exit: (12) has_a_property(sword, has_edges, two_edges) ? creep
Property = has_edges,
Value = two_edges ;

[snip]

  Redo: (12) has_a_property(sword, _41916, _41918) ? creep
  Call: (13) is_a(sword, _70666) ? creep
  Exit: (13) is_a(sword, bladed_hand_weapon) ? creep
  Call: (13) has_a_property(bladed_hand_weapon, _41916, _41918) ? creep
  Call: (14) has_a(bladed_hand_weapon, _41916, _41918) ? creep
  Exit: (14) has_a(bladed_hand_weapon, attack_type, cut) ? creep
  Exit: (13) has_a_property(bladed_hand_weapon, attack_type, cut) ? creep
  Call: (13) has_a(sword, attack_type, _76126) ? creep
  Fail: (13) has_a(sword, attack_type, _77016) ? creep
  Redo: (12) has_a_property(sword, attack_type, cut) ? creep
  Exit: (12) has_a_property(sword, attack_type, cut) ? creep
Property = attack_type,
Value = cut ;

[snip]
```

This trace shows Prolog using recursion and temporary variables:

- Checking direct properties of 'sword' – such as 'two_edges'.
- When those run out it climbs the is-a hierarchy.
- At each level, it collects inherited properties. – such as attack_type from bladed_weapon.

Using a specific instance of a sword shows how inheritance works when a more specific property overrides more general inherited values. Note from the above output the property 'era_used' for a sword is inherited from the root 'weapon' class.

Long swords were introduced in 1250:

```
?- has_a_property(long_sword, era_used, When).
When = (1250, 1550)
```

However, the specific instance of a long sword belonging to William Wallace has a more specific use date:

```
?- has_a_property(the_wallace_sword, era_used, When).
When = 1298 ;
```

When all the properties for The Wallace Sword are listed, the more specific date overrides both values era_used applied at the root of the tree and applied to all long swords. This list shows a mixture of local, inherited and overridden properties:

```
?- list_properties(the_wallace_sword).
Properties for the_wallace_sword:
-----
- alternate_name = bastard_sword
- alternate_name = great_sword
- alternate_name = hand_and_a_half_sword
- ammunition_type = no_ammunition
- attack_type = thrust
- effective_against = personnel
- effective_range = close_quarters
- era_used = 1298
- has_edges = two_edges
- has_part = fullers_aka_blood_grooves
- historical_origin = scotland
- is_throwable = not_throwable
- length_inches = 40.48
- movement_speed = fast
- people_reqd = one_person
- primary_material = steel
- use_case = attack
- used_by = william_wallace
- weight_kg = 2.7
```

The underlying `has_a_property` rule implements recursive inheritance with local overrides. The cut operator (`\+`) is used to stop general properties from being included when a more specific instance is available.

Property overrides demonstrate the difference between traditional monotonic and non-monotonic logic. Property overrides introduce non-monotonic behaviour. When the specific era for the Wallace sword is defined as 1298, it supersedes the inherited date from the sword class. This allows the network to handle specific exceptions without undermining the general rules of network inheritance.

It is also possible to ask some specific questions of the network using rules like `longer_range`:

```
?- has_a(biological_weapon, effective_range, Value).
Value = long.

?- has_a(long_bow, effective_range, Value).
Value = long.

?- has_a(dagger, effective_range, Value).
Value = close_quarters.

?- longer_range(long_bow, biological_weapon).
false .

?- longer_range(long_bow, dagger).
true .
```

As the `effective_range` property is a linguistic variable rather than a numerical value, the following weighting facts were used to convert to numbers and allow direct comparison in the `longer_range` rule. The specific values do not matter as this is only to allow comparison of these terms.

```
% Higher number = longer range
range_to_weight(long, 3).
range_to_weight(medium, 2).
range_to_weight(short, 1).
range_to_weight(close_quarters, 0).
```

One final example shows the output of the `show_tree` rule, which gives each of the weapons and their level in the hierarchy:

```
?- show_tree(weapon).
-- weapon (0)
  -- melee_weapon (1)
    -- bladed_hand_weapon (2)
      -- dagger (3)
        -- anelace (4)
        -- poingnard (4)
        -- rondel (4)
        -- stiletto (4)
      -- knife (3)
      -- sword (3)
        -- arming_sword (4)
          -- almace (5)
        -- broad_sword (4)
        -- falchion (4)
          -- cleaver_falchion (5)
          -- cusped_falchion (5)
          -- medici_falchion (5)
        -- long_sword (4)
          -- the_wallace_sword (5) (..etc..)
```

Ethical Considerations

The network design considerations made for this project would be valid for most subject matter. The implementation in both python and Prolog languages has been made in as generic manner as possible. This would have allowed for almost any subject to have been used to give example to the construction of a semantic network.

While there is nothing inherently malicious about using historic articles of warfare as an example, ethical considerations arise around the use of a medieval weaponry dataset to study semantic networks and programming in Prolog. Switching to non-violent objects can make the study of computer science more welcoming to those who might otherwise feel alienated or intimidated by this dataset.

Historical sources like the Metropolitan Museum of Art (1991) treat these items as artefacts of technical and cultural evolution. Humans can understand the historical context, whereas AI systems lack contextual awareness, and may struggle with adequately weighting non-aggressive features. Moreover, if an AI agent were to use this network as a knowledgebase to solve a problem, it is unlikely to have the human understanding needed to distinguish museum artefacts from functional weapons.

Furthermore, if the output of this exercise were to be fed into neural network models, there is a danger that the system may learn embedded bias, such as predicting or advocating violence. Worse, there is a risk that this aggression might be internalised in a 'black-box' manner making such bias harder to detect and mitigate.

[1883 words]

Part 2: Connectionist Methods

In 1943, Warren McCulloch and Walter Pitts proposed that the brain works like a series of logic gates (0's and 1's). They proved that neural activity is a biological way of doing maths. Modelling this behaviour in a digital neural network is a well-known problem-solving approach. In this section, I will discuss the creation and performance of several neural networks aiming to help 'a bank' determine which customers to call with a view to making a deposit with the bank.

Dataset Preparation

The UCI 'Bank Full Dataset' is related to the phone based direct marketing campaigns of a Portuguese bank from May 2008 to November 2010. Customers were often contacted multiple times to assess whether they would subscribe to the deposit product. (Moro 2014)

The dataset includes 17 inputs, ordered by date and contains just over 45,000 rows. The classification goal of my Multi-Layer Perceptron (MLP) is to predict if the client will subscribe or not.

Exploratory Data Analysis (EDA) was carried out, and the dataset was audited for quality and consistency. While not every feature is discussed here, all were examined individually during EDA, as shown in the accompanying Jupyter notebook. A variety of techniques were required to prepare the broad range of data.

A Biased Dataset

Exploring the target dependent variable 'y' (Figure 4) shows that there is considerable bias in the dataset. While the success rate is actually very good for a cold calling telephone marketing campaign (Cognism 2025), most people did not deposit funds.

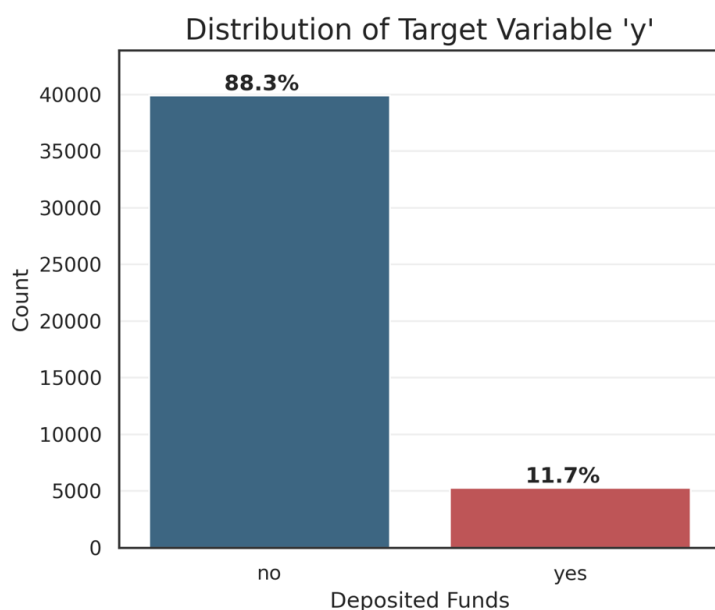


Figure 4 Showing that the dataset is highly biased towards negative results.

This bias causes issues for the neural network because the weights will tend to shift to favour negative responses; there are not many instances of positive responses to learn from. The network is likely to learn to predict 'no' whatever the input because this is most often correct.

Other features were assessed individually for usefulness and potential negative impact on model performance.

Outliers

Four features that were likely to contain outliers and/or very wide distributions: age, balance, campaign and previous [number of times contacted].

Age

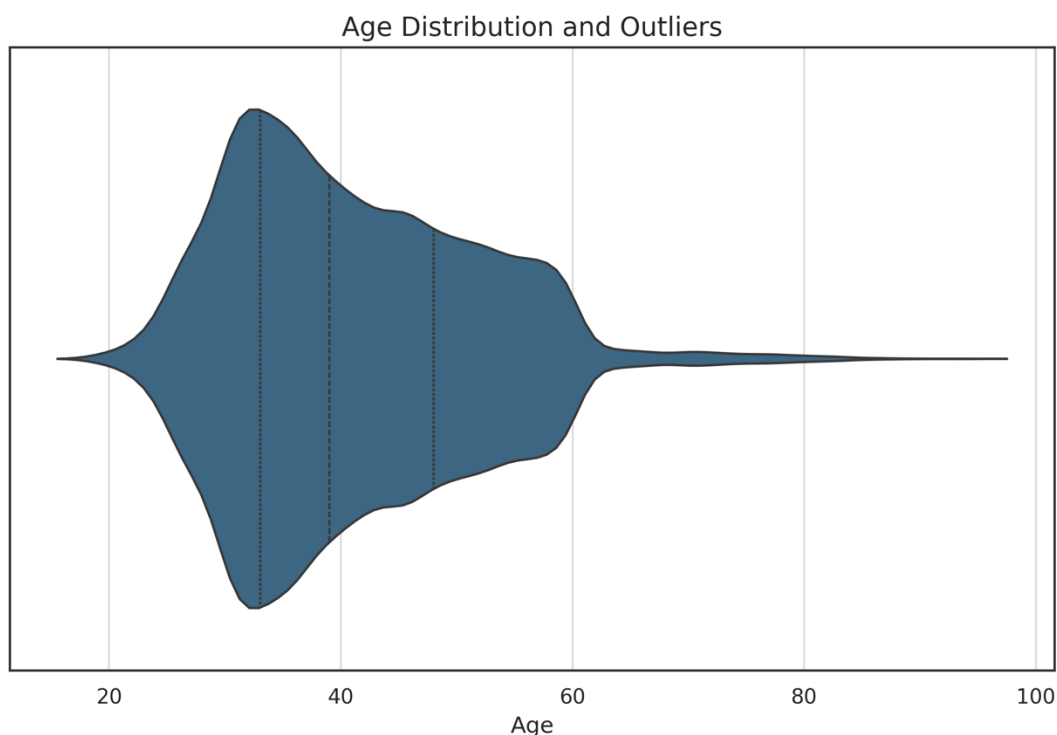


Figure 5 Violin plot for age, showing distribution and outliers

The plot shows a massive bulge between 30 and 45. This is expected and is the target audience for the bank's marketing department. The tail shows the older outliers who are people over 65-70 years old.

A Z-Score scaler was used to encode all features with outliers.

Education

While the dataset is largely complete, there are several features where there are 'unknown' values. These were identified and handled as behavioural categories.

The unknown group in the education feature had a success rate of 13.6%, which is higher than the dataset average (11.7%). Thus, unknown was treated as a specific feature rather than being dropped. It was mapped to sit between secondary and tertiary education, preserving the ordinal meaning of educational progression. A numeric version of the feature was engineered to allow scaling.

Contact Type

The contact feature relates to the type of medium used to contact the individual.

This is either landline telephone, cellular phone or unknown. While mobile phones dominate communications today, this was less true in 2010. Since there is a significant group of unknown values, using one-hot encoding allows the model to learn their unique signature without my needing to guess if they 'should have been' cellular or landline contacts.

Pdays – Days Since Previous Contact

The pdays feature required special consideration. People who have never been called before are represented as -1, while the rest of the data shows how long it has been since they were contacted previously. The uncontacted make up most of the data:

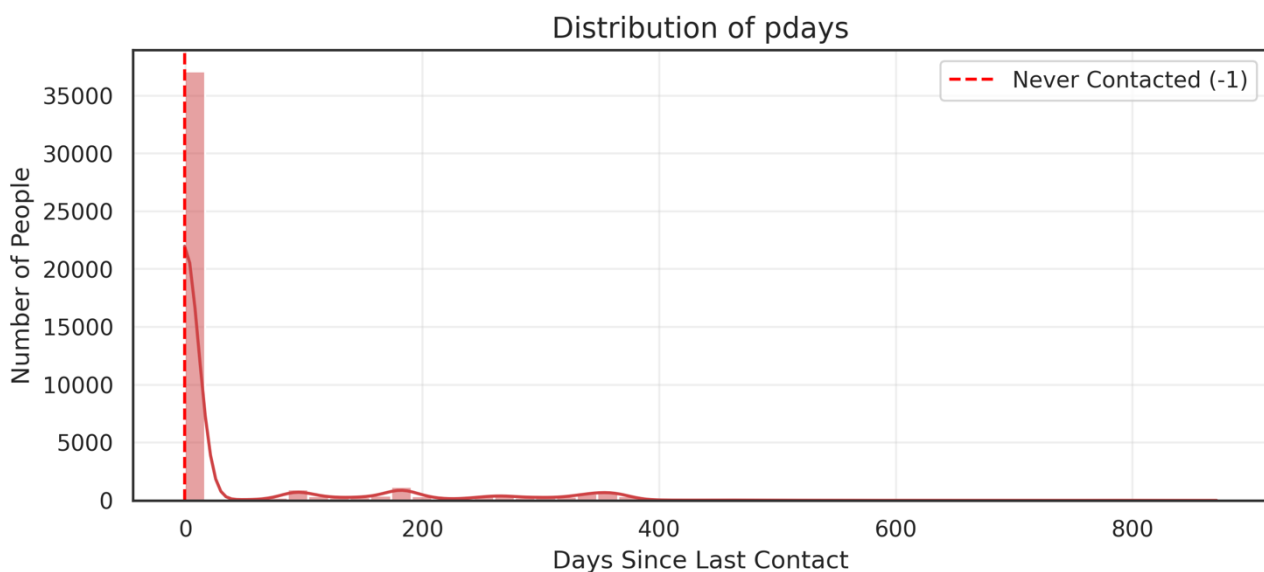


Figure 6 Showing how most of the data relates to the uncontacted.

When the -1 values are filtered out, the pdays distribution is multi-modal, exhibiting several local maxima. There are peaks at approximately 100, 180, 275 and 365 days. This suggests a re-engagement strategy based on quarterly cycles.

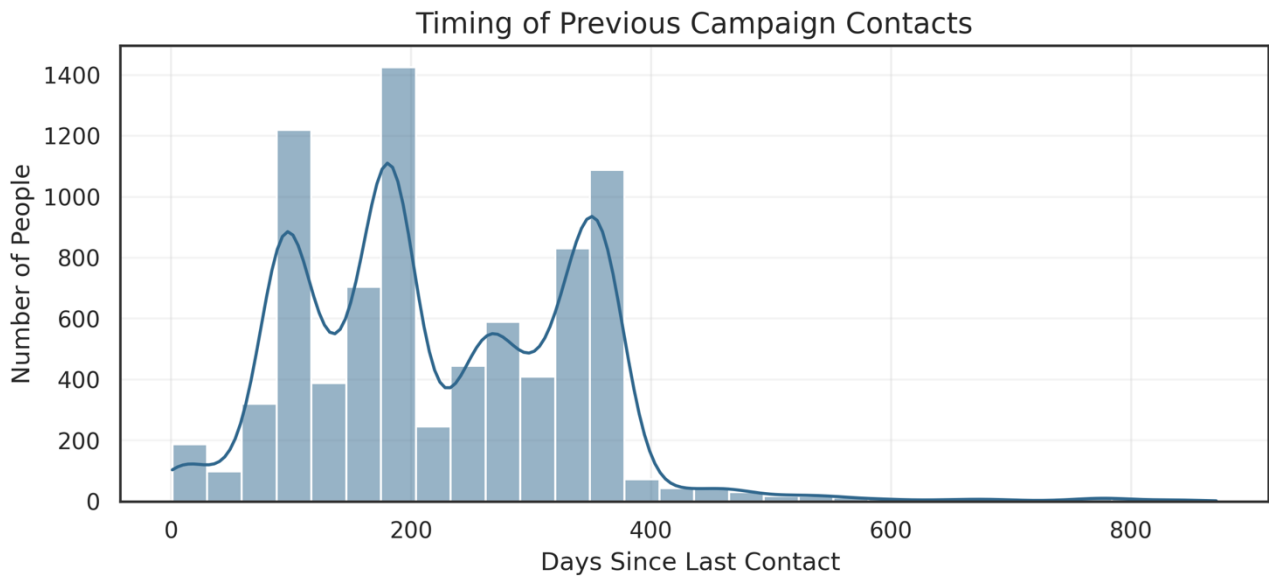


Figure 7 Distribution of filtered pdays feature (-1 removed)

To ensure the MLP captures these specific windows of time, I used a binning strategy based around these values:

Bin Value	Range
0	-1 (never Contacted)
1	< 30 days
2	< 150 days
3	< 250
4	Longer term

Previous Outcome

It makes sense that the 'previous outcome' feature would likely be a useful indicator for predicting the future outcome.

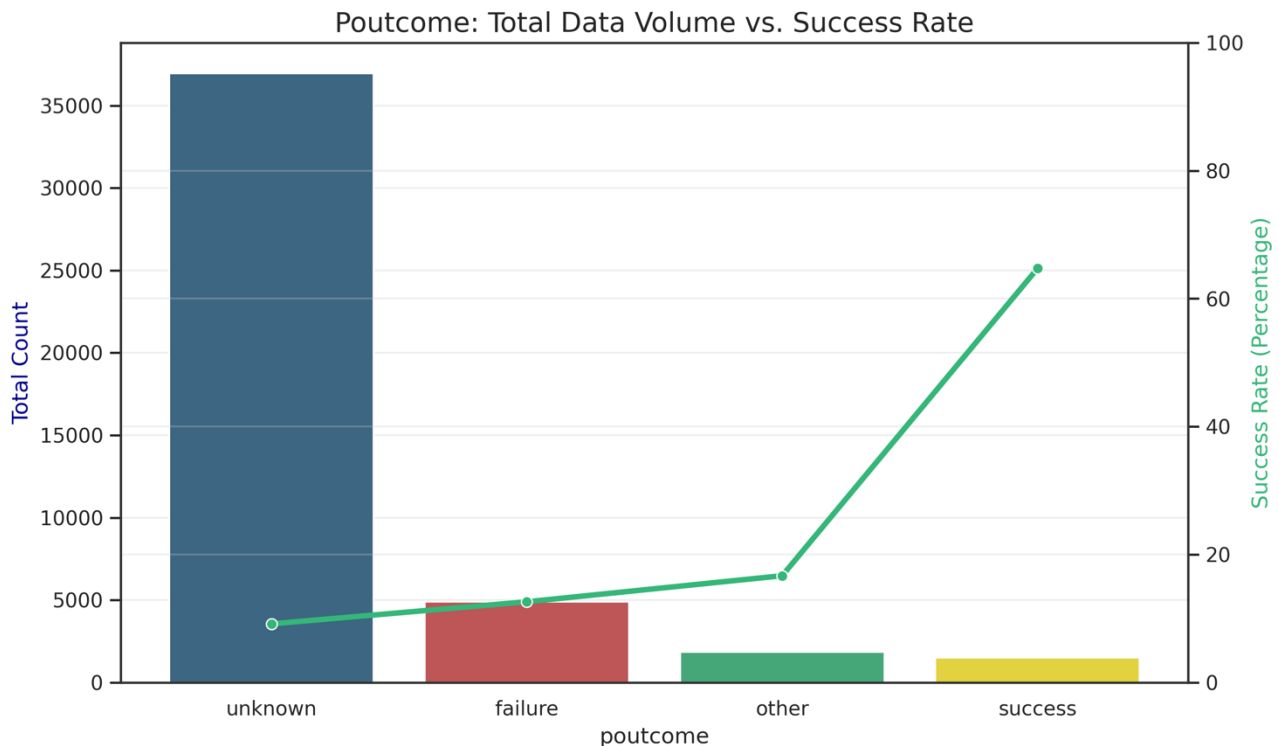


Figure 8 Exploring previous outcome

The graph shows that there is a massive disparity in the results, and if an individual said 'yes' last time, they are overwhelmingly likely to say 'yes' again.

One Hot Encoding would allow the MLP to determine for itself that the previous outcome is an excellent indicator. However, because the number of unknowns is high, the MLP might miss these rare but important success signals, especially once the data is split across training, validation and test sets.

To assist the MLP in spotting these rare but strong indicators, I used a map to manually weight these values highly to reflect their importance:

Value	Weighted Map
Success	7
Other	2
Failure	0
Unknown	1

Encoding and Scaling

All features were encoded as numeric values and scaled to give small floating-point values, generally between -1.0 and 1.0, depending on the encoding method used.

This table shows how each included feature was handled.

Encoding	Features
Binary	'housing', 'loan', 'y'
Z-Score Standardisation	'age', 'balance', 'campaign', 'previous'
Min-Max Scaling	'pdays_binned', 'month_numeric', 'education_numeric', 'poutcome_numeric'
One Hot Encoding	'job', 'marital', 'contact'

One-Hot Encoding

The non-ordinal categorical features were expanded using One Hot Encoding. This prevented the model from assuming a false linear relationship between unrelated categories for example "Single" vs. "Married". These categories have no inherent order, so one-hot encoding ensures the MLP treats each value as distinct.

Binary Variable Encoding

Binary features were simply encoded as 'yes' = 1.0 and 'no' = 0.0.

Normalisation Functions

Z-Score Standardisation was applied to features with significant outliers, such as balance and campaign because this centres the values around 0 as the mean. The standard deviation is used to calculate the spread of values.

The function for this operation is: $x_{scaled} = \frac{x - \mu}{\sigma}$

Min-Max Scaling was used for other numeric features to squash values into a 0 to 1 range.

The function is: $x_{scaled} = \frac{x - \min(x)}{\max(x) - \min(x)}$

Removal of Problematic Features

Several features were removed from the model. The 'duration' feature is the most interesting problematic feature because as well as being highly skewed by outliers, it also allows for data leakage 'from the future' into the model.

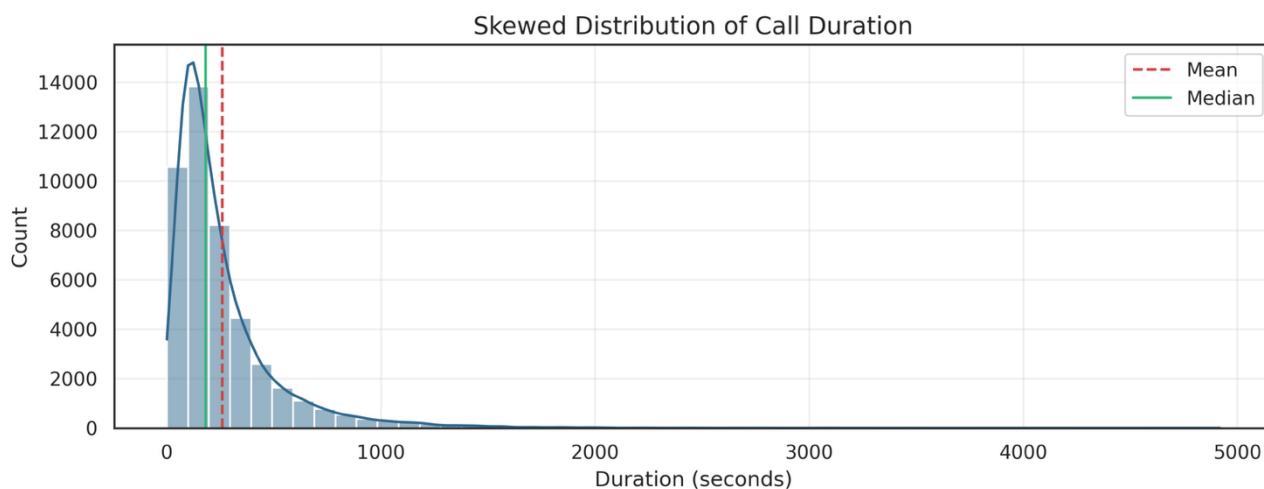


Figure 9 Exploring duration

If a person talks for 30 minutes, they have almost certainly already said "Yes". Conversely, if they end the call after a very short time, then they are likely to have said "No". The longer duration is a result of positive intent, not a cause.

As duration is only known after the call, it cannot be used to decide who to call tomorrow. Including it would leak target information into the model.

Other features were also dropped for the following reasons:

Feature	Action	Reasoning
Duration	Dropped	Target leakage; duration is not known at the time of prediction.
Default	Dropped	Near-zero variance; 98% of entries are 'no'.
Day	Dropped	Does not appear to add value; no obvious trends.
Pdays	Binned Version Kept	Re-encoded to handle multi-modal distribution and -1 values.
Month	Encoded version kept	Re-encoded as values 1-12
Education	Encoded version kept	Re-encoded as values 0-3
Poutcome	Encoded version kept	Re-encoded as values 0-7

Correlation with Dependent Variable 'y'

The remaining features in the dataset were compared with the target variable 'y' to show which features have the best correlation. These correlations indicate which features the MLP is likely to learn from most strongly.

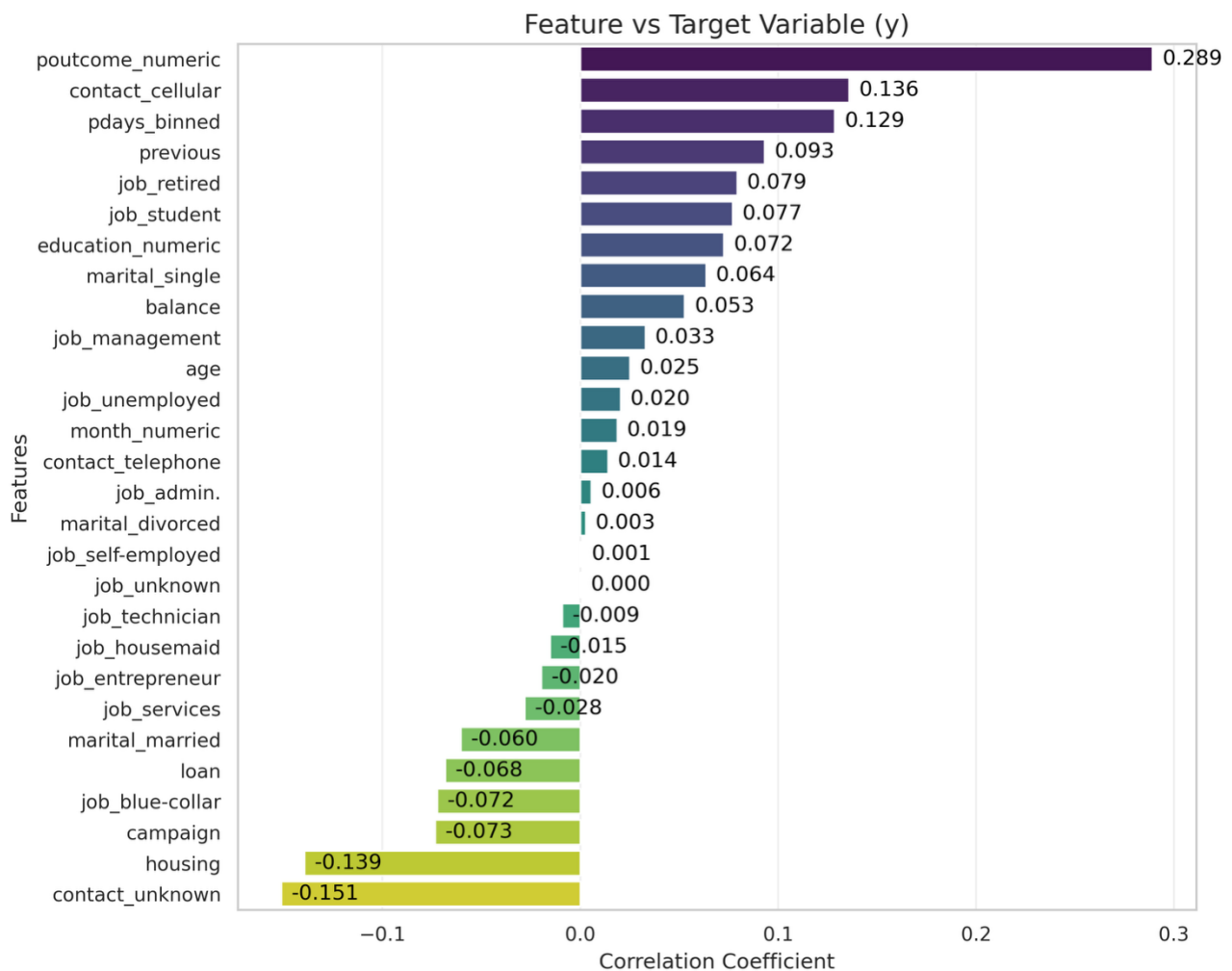


Figure 10 Correlation of features vs 'y'

Strongest Positive Correlation: poutcome_numeric (0.289), contact_cellular (0.136) and pdays_binned (0.129). The previous outcome was already shown to be a strong indicator but contact by cellular phone is a slight surprise.

Demographics: job_retired, job_student, and marital_single are all positively correlated with success. This confirms that specific life stages are more likely to say "yes". Students, for example, are likely planning for their future and trying to make savings, while the retired likely have spare cash to invest.

Weak correlation: The balance (0.053) correlation is weaker than might be expected, meaning it is not the primary driver for a 'yes' result in this campaign.

Post Processing

After preprocessing, the dataset was written out to a new CSV file to allow the same data to be used in both the MLP written 'from scratch' and the MLP based on Scikit-Learn Python libraries.

Before loading the data into either MLP implementation, one further step was required: splitting the dataset into training, validation and testing data. The data was split 80% towards the training data, with 20% going to form the test dataset. The training data was split further, again 80:20, into training and validation sets.

The dataset is date ordered, meaning that it must be shuffled to avoid the MLP learning patterns based on the sequence of the rows, such as time-of-year trends.

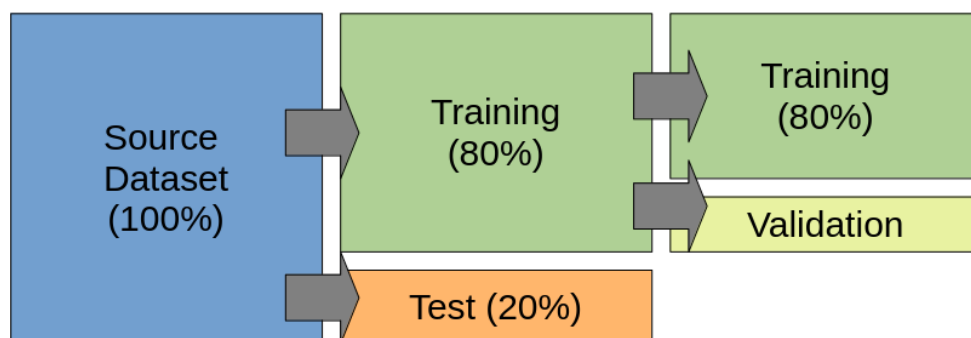


Figure 11 Training, validation and test data splitting method.

Furthermore, 'stratified sampling' of the success and failure classes means that the target variable imbalance (88.3% vs 11.7%) was preserved across the training, validation and testing datasets. This prevents the result from being skewed by a disproportionate distribution of target variables in these datasets.

In the instance of the 'from scratch' MLP, this was done using a custom python function, whereas splitting the data for the Scikit-Learn based MLP utilised the 'train_test_split' function with the stratify and shuffle parameters enabled.

Implementation of ANN From Scratch

The 'from scratch' MLP was implemented in Python as a Class 'MLP_From_Scratch'. It provides several public functions:

Function	Purpose
Class Constructor	Creates network using provided dataset. Splits into test, train and validation sets. The number of input neurons is determined automatically. Allows user to tune parameters: • no_hidden_neurons • no_output_neurons • output_threshold • validation_split (%)
train_network	Performs network training for a user supplied number of epochs. Allows user to define learning_rate. Steps: • Feed Forward • Back Propagation (Output Layer) • Back Propagation (Hidden Layer) • Weights and biases updated
evaluate	Return accuracy.
plot_error	Plots the training and validation set losses for comparison.
plot_confusion_matrix	Plots a graphical confusion matrix.
print_confusion_matrix	Prints a text representation of the confusion matrix.
save_weights	Save out the weights and biases for future use.
load_weights	Load in a set of saved weights.
get_report	Print a report of model accuracy, recall, precision and F1-score.
set_model_parameters	Allows user to reset learning_rate and output_threshold hyper-parameters.

Network Architecture

After encoding, the input layer consists of $N=28$ neurons. The hidden layer is explicitly defined as 10 neurons.

As this is a binary classification, the output layer consists of 1 neuron using a Sigmoid activation function, outputting a probability score.

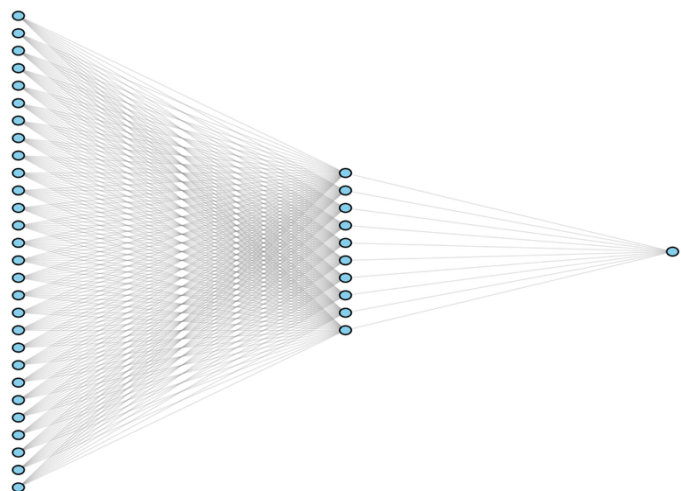


Figure 12 Neural network architecture

Feed Forward Pass

The forward pass is simply computing the dot product of weights and inputs, plus the biases

$$Z = w \cdot x + b$$

and then applying activation functions at the hidden and output layers.

The ReLU (Rectified Linear Unit) function is used in the hidden layer. Initially, Sigmoid was used for both layers, but performance improved significantly when ReLU was applied to the hidden layer. The network likely experienced a vanishing gradient due to the limitations of the sigmoid derivative function – the weight values become very small, very quickly.

ReLU Activation Function

$$ReLU(x) = \max(0, x)$$

Sigmoid Activation Function

$$A = \sigma Z$$

where $\sigma(x)$ is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The Sigmoid function is the standard choice for the output layer in a binary classification task because it squashes any input into a value between 0 and 1, which represents the probability of an individual subscribing. (Bishop 2006, pp236) The default threshold for the output later was set at >0.5 = 'yes'.

Back Propagation

The backwards pass works by calculating the partial derivative of the loss with respect to each weight.

Loss functions are used instead of raw error because raw errors can cancel each other out. This can give the illusion of good performance even when the model is doing poorly. A proper loss function means that all mistakes contribute to the total loss, ensuring the network gets a workable gradient to learn from.

Mean Squared Error:

$$MSE = \frac{1}{m} \sum_{i=1}^m (y^i - \hat{y}^i)^2$$

Originally, I had used MSE to calculate the loss in the back propagation passes. However, performance was poor, so alternative loss functions were explored.

Following the recommendations in Bishop, the loss function was transitioned from MSE to Binary Cross-Entropy, improving results.

Binary Cross-Entropy:

$$BCE = \frac{-1}{m} \sum_{i=1}^m [y^i \ln(y^{\wedge i}) + (1 - y^i) \ln(1 - y^{\wedge i})]$$

MSE assumes the data follows a normal distribution, whereas BCE works with data that follows a binary output, in this case 'yes' or 'no', in a Bernoulli distribution (Deisenroth 2021). It also avoids the gradient flattening associated with Sigmoid, preventing learning from slowing over time.

Weight and Bias Update Step

The final step of the learning process is where the gradients calculated during backpropagation are used to adjust the weights and biases.

This model employs Gradient Descent, governed by the following update rules:

$$\begin{aligned} w_{new} &= w_{old} - (\eta \cdot \partial L / \partial w) \\ b_{new} &= b_{old} - (\eta \cdot \partial L / \partial b) \end{aligned}$$

The 'learning rate' determines the step size taken toward the local minima. Early testing used a learning rate of 0.05, which caused loss graph line divergence as weight updates overshoot the minima. Through experimentation, a lower rate of 0.005 was selected.

While weights determine the strength and direction of the relationship between input features and the prediction, biases provide the model with flexibility. By adding a bias, the activation function can shift its position on the graph away from the origin. This is crucial because real-world data patterns rarely start at zero; the bias allows the neural network to move its decision boundary for the specific dataset.

As only 11.7% of customers subscribed, the bias is particularly important. It reflects the skewed reality of the dataset. It forces the model to be pessimistic, only predicting a 'yes' when the evidence is overwhelmingly positive in the features.

Performance Evaluation

To monitor the model's ability to work with as-yet unseen data, the training of the model was validated every 100 epochs by using a validation dataset. This is split from the supplied training dataset automatically on model initialisation.

The validation set was used to monitor the performance during training, or to adjust the number of epochs or which learning rate to use. The Binary Cross-Entropy loss was calculated for both the training and validation subsets.

Using the test dataset at this stage would be leaking information into the training process. This risks producing a model that performs well on the test set but poorly in real-world use. Using a separate validation set keeps the test set completely unseen until it is used in the post-training phase.

The training losses are stored and plotted on a graph after the training is completed. The point where learning stops is visible in the curve.

The following graph shows how the losses flatten out over 10,000 epochs after an initial steep learning period. Further training brings diminishing returns and costs unnecessary compute time.

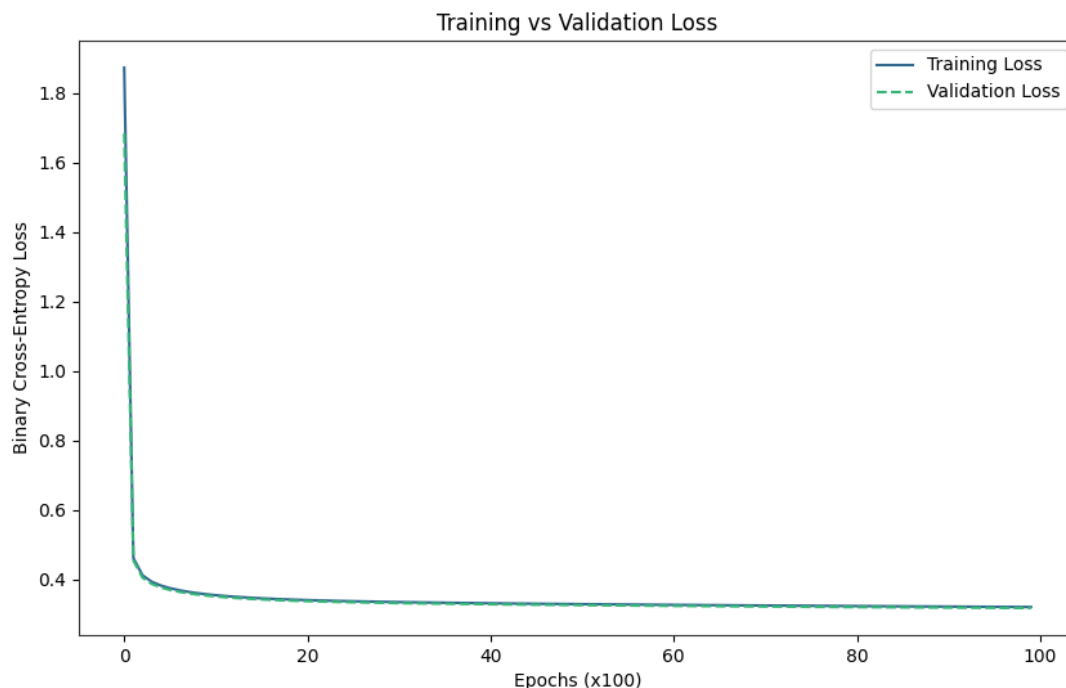


Figure 13 MLP Training vs Validation loss over 10,000 epochs

After training, a confusion matrix is drawn to show the true behaviour of the MLP at predicting against the test dataset.

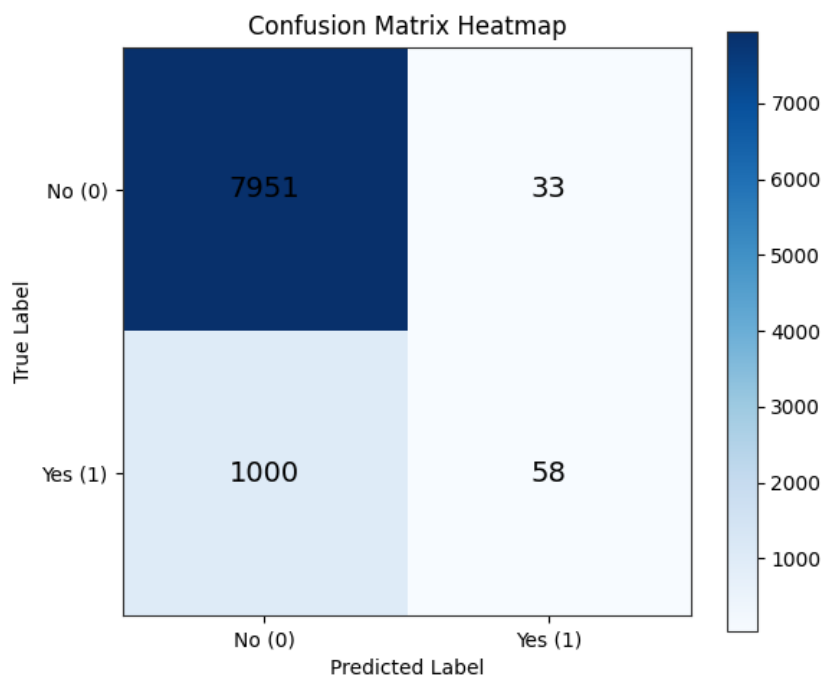


Figure 14 Confusion Matrix, Output Threshold = 0.5

True Negatives: The model is excellent at identifying people who will not subscribe. This is expected because 'no' is the majority class.

True Positives: The model correctly identified only 58 successful deposits.

False Positives: The model predicted a 'yes', but it didn't happen. This number is quite low, which means that the bank wouldn't waste much effort if they called these people and were told 'no'.

False Negatives: There were 1000 people who said 'yes' but the model predicted they would say 'no', meaning that the bank would be unlikely to expend the effort to call them. This would mean that the bank missed out on many deposits.

Using the above values, the following metrics were calculated:

Metric	Score	%
Accuracy	0.8858	88.6%
Precision	0.6374	63.7%
Recall	0.0548	5.5%
F1-Score	0.1010	10%

Metric	Mathematical Formula	Description	What it tells the Bank
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	The proportion of total predictions that were correct.	Overall success rate.
Precision	$\frac{TP}{TP + FP}$	The proportion of positive identifications that were correct.	When we call someone a potential customer, how often are we right?
Recall	$\frac{TP}{TP + FN}$	The proportion of actual positives that were identified correctly.	Out of everyone who would have said 'Yes', how many did we find?
F1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	The mean of Precision and Recall.	The balanced score.

The confusion matrix reveals that while the MLP achieves a high accuracy of 88.6%, it suffers from a significant number of False Negatives. The model is heavily biased toward the majority class 'no', resulting in a very low recall of 5.7%. This indicates that the model defaults to the most frequent outcome. This was expected during the data preparation phase.

While the bias is correctly shifting the activation function to reflect probability of the majority 'no' class, a standard 0.5 classification threshold was too high. The model was tuned using the output threshold to determine whether these scores could be improved.

The output threshold was varied from 0.0 to 1.0 in a loop, and the performance metrics calculated. These results were plotted on the following graph:

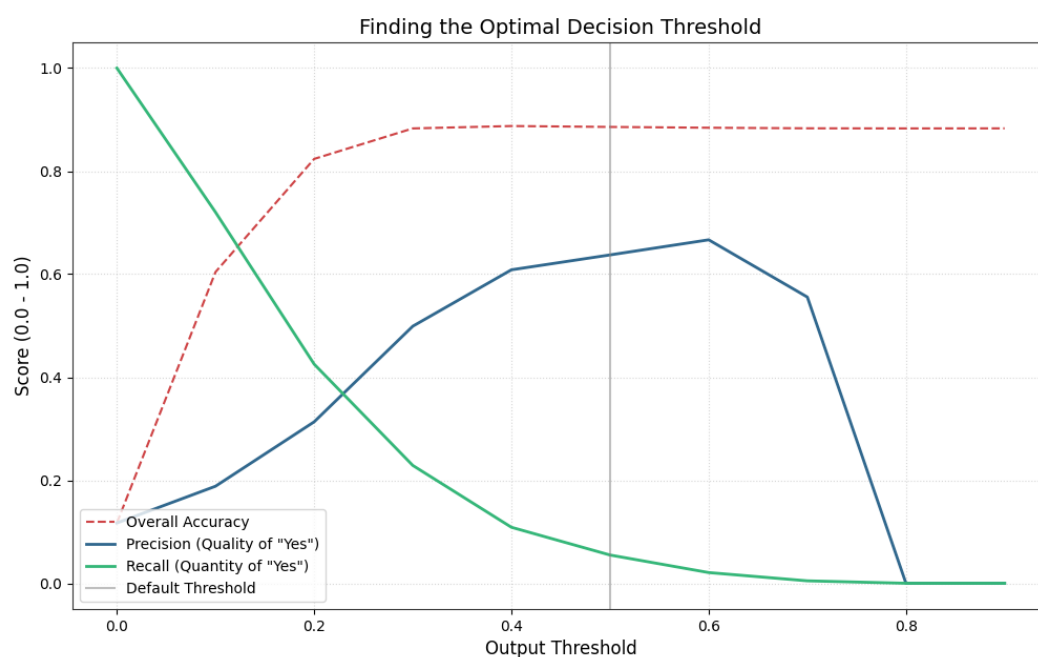


Figure 15 The cross over point of recall and precision is the best compromise.

The point at which the recall and precision lines cross gives the best trade-off between these scores, calculated to be 0.23.

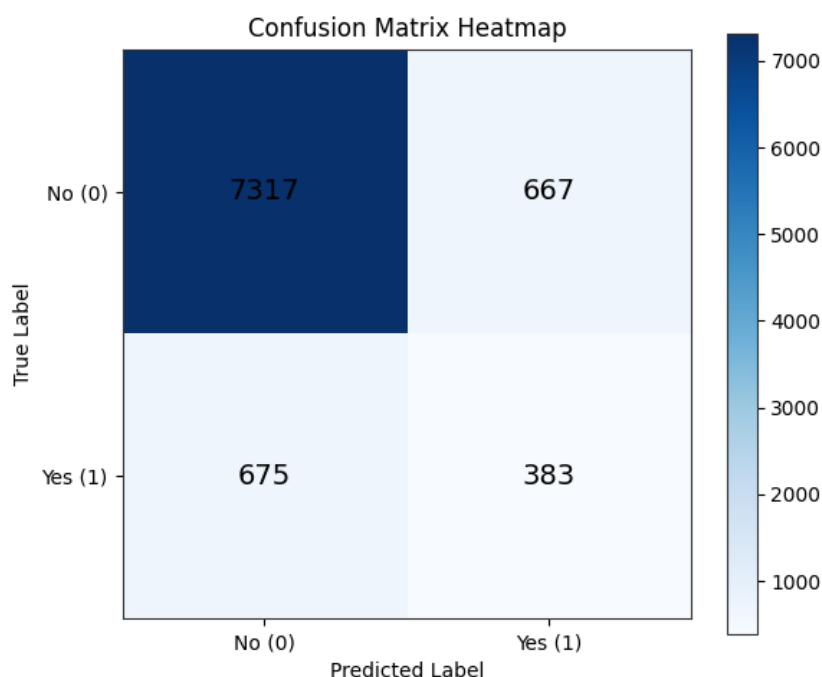


Figure 16 Confusion Matrix, Output Threshold = 0.23

This gives a much more balanced result.

Metric	Score	%
Accuracy	0.8516	85.1%
Precision	0.3648	36.5%
Recall	0.3620	36.2%
F1-Score	0.3634	36.3%

Lowering the threshold seems to contradict the role of the bias but allows me to place more priority on the recall of the model. By adjusting the output threshold, the recall improved dramatically from 5.4% to 36.2%. Accuracy and precision decreased slightly but the model became more useful for a bank marketing campaign.

In this context, the cost of a phone call resulting in a False Positive is relatively low compared to the high value of a 'True Positive'. This ensures the bank will contact more potential customers. Adjusting the threshold is a business decision made after training.

ANN Implementation with Libraries

An MLP was also developed using the Scikit-Learn library to act as a comparison against the 'from scratch' version discussed above.

Choice of Library

The pre-processed dataset was split 80:20 into training and test; validation being internal to the model. I used the Scikit-Learn library 'train_test_split' function with the stratify and shuffle parameters enabled. Keras does not provide data-splitting utilities, so Scikit-Learn was the more convenient choice.

I also chose to use Scikit-Learn rather than Keras because the dataset is only moderately sized. Once engineered, it has 28 input features and around 45,000 rows. Keras is built by Google and is designed for large-scale deep learning projects. (Keras.io 2026)

Keras requires the user to define input shapes explicitly and needs to have access to specific back-end libraries, such as TensorFlow, which proved challenging to install on a workstation. Scikit-Learn handles the underlying maths and provides a simple interface without needing further libraries beyond NumPy.

Model Parameter Configuration

The MLPClassifier class was used to create the neural network.

To ensure a fair comparison, I chose parameters for the network that would be as close to the 'from scratch' configuration as possible:

From Scratch MLP Setting	MLPClassifier Setting
10 hidden neurons	hidden_layer_sizes = (10,)
Hidden layer activation: ReLU Output layer activation: Sigmoid	activation = 'relu'
Weight updates: gradient descent	Solver = 'sdg'
Default Learning Rate = 0.005	learning_rate_init = 0.005
Epochs = 10,000	max_iter = 10000

This aims to ensure that comparison between the models will be based on the implementation, rather than the underlying mathematical functions in use.

Performance Comparison

As before, a confusion matrix was drawn to show the true behaviour of the MLP at predicting against the test dataset. The test dataset accounts for 9043 people:

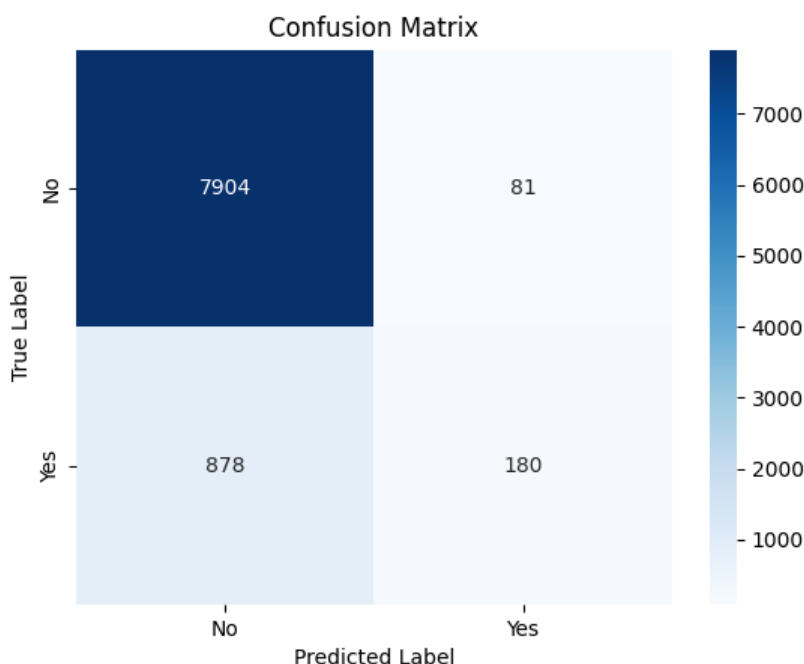


Figure 17 Confusion Matrix, SKL implementation

The performance of the library-based model was better than the base ‘from scratch’ MLP in several ways.

	From Scratch, TH=0.5	From Scratch, TH=0.23	SKL MLP
True Negatives (TN)	7,951	7,317	7,904
True Positives (TP)	58	383	180
False Positives (FP)	33	675	81
False Negatives (FN)	1000	667	878

In both implementations, the number of both true and false negatives was high, as predicted due to the skewed nature of the dataset.

The TH=0.5 model is very conservative, predicting ‘no’ in most cases. The number of False Positives is very low, so when it does predict success, you can usually believe it. However, it misses most actual positives.

The TH=0.23 model is much better as a marketing tool. It prioritises differently, allowing it to catch many more positive results. This has come at the cost of a large increase in False Positives. As noted previously, the cost of calling a non-subscriber is low, and the number of deposits gained would be significantly higher.

The SKL model has much better internal optimisation. It found more True Positives than the TH=0.5 model and has a low False Positive rate. The 'from scratch' models took around a minute of CPU time to train; SKL took 3 seconds to reach this level of prowess. It implements an 'early stopping' monitor meaning that 10,000 epochs weren't needed. It would likely perform even better if tuned to the same 0.23 threshold.

Metric	From Scratch, TH=0.5	From Scratch, TH=0.23	SKL MLP
Accuracy	88.58%	85.16%	89.35%
Recall	5.48%	36.48%	17.49%
Precision	63.74%	36.20%	67.03%
F1-Score	10.09%	36.34%	27.74%

The SKL MLP achieved the highest overall accuracy. However, in the business context of the dataset, accuracy is not the correct metric to use alone; it still misses 878 potential subscribers.

The TH=0.23 model is the clear winner with respect to the bank's marketing requirements. It improved the Recall Score from a terrible 5.48% to a winning 36.48%. This represents an increase of around 6.5 times in the number of successful responses.

The SKL MLP has the best Precision, meaning the "yes" predictions are the most reliable. However, the cost of calling a customer who says "no" (a False Positive) is generally much lower than the value of missing a customer who would have said "yes", so it is not the best model from this perspective.

Advantages and Disadvantages

One disadvantage is that the library does not expose how the validation set is composed or allow me to graph validation losses. This reduces visibility of the training process. While early stopping is internally implemented, it is not possible to create a visual check against over fitting.

Furthermore, I have complete control over the validation dataset in my model, allowing me to analyse its content if needed. I can also access the weights and biases at any given time, even *during* training.

Another disadvantage of the library MLPClassifier is rigidity in the implementation. The configured activation function cannot be varied between layers. Conversely, the 'from scratch' implementation is more modular. It allows for flexibility and experimentation with different activation functions at different levels.

The main disadvantage of using the 'from scratch' model is the length of training time. Many thousands of Python loops over matrix calculations are very slow, even though NumPy itself

is internally efficient. The Scikit-Learn library uses highly optimised C/C++ code in the background.

Developing the model from scratch provided control and transparency to monitor the impact of class imbalance in the dataset by plotting the validation loss, and by modifying output thresholds accordingly. Using a library provides speed, built-in optimisation and convenience.

Parameter Tuning and Model Optimisation

GridSearchCV

A lot of the optimisation steps applied so far have been ad hoc and based on observation and educated guesses. To move beyond manual trial-and-error adjustments, GridSearchCV from Scikit-Learn was used to systematically tune hyper-parameters in the SKL based model.

I have already identified that accuracy is not the best metric for this dataset, so the optimisation target was set to Recall to combat class imbalance in the target variable.

CV stands for Cross-Validation, which means that the data is rotated so that each row has the chance to be part of the training and validation datasets. I have used $cv=3$, which defines three 'folds' of the data. This means that the model is trained three times, using the data folds as follows:

Training	Validation
1 + 2	3
1 + 3	2
2 + 3	1

This method reduces any errors that might be derived from the data itself, such as the lucky inclusion of 'easy' rows which might contain hidden shortcuts to the target variable. An example of such a shortcut would be the duration feature, which was removed entirely during preprocessing.

I defined the following grid of parameters to extensively test different model architectures, learning rates and weight/bias adjustment algorithms:

```
'hidden_layer_sizes': [(10,), (15,), (10, 5), (15, 5)],  
'learning_rate_init': [0.001, 0.01, 0.005, 0.03, 0.05],  
'max_iter': [2000, 5000, 10000, 15000, 20000],  
'solver': ['sgd', 'adam', 'lbfgs', 'newton-cg']
```

This is an extensive search grid. To optimize the hyper-parameter search, the `n_jobs` parameter was set to -1 to use all the cores in my workstation. By distributing the Grid Search across all CPU cores, the computational time was reduced. This allowed for a more exhaustive search of the parameter grid than would be feasible otherwise. Even so, this grid took 14 minutes to complete.

The best parameters were found to be:

```
'hidden_layer_sizes' = (15, 5)  
'learning_rate_init' = 0.05  
'max_iter' = 2000  
'solver' = 'adam'
```

The grid search revealed that the 'Adam' optimizer and two hidden layers of 15 and 5 neurons provided a more stable descent than the model defined previously. The Adam

optimiser changes learning rate for individual parameters, which is useful with this dataset as the impact of each feature on the model varies significantly (GeeksForGeeks 2025).

Running the test data through the best model produces the following confusion matrix:

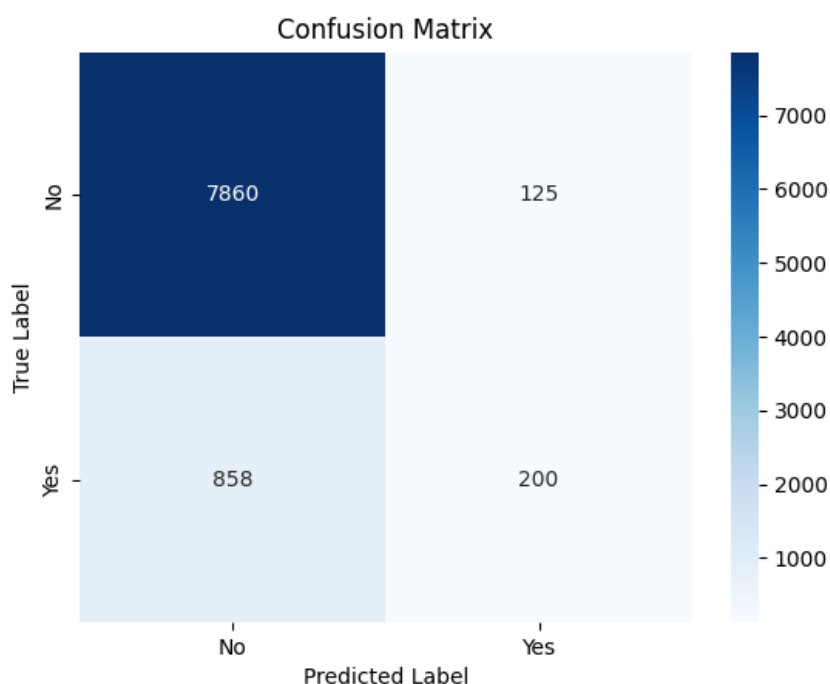


Figure 18 Confusion matrix, grid search optimised MLP

Adding these results to the table:

	From Scratch, TH=0.5	From Scratch, TH=0.23	SKL MLP	Grid Optimum
True Negatives (TN)	7,951	7,317	7,904	7,860
True Positives (TP)	58	383	180	200
False Positives (FP)	33	675	81	125
False Negatives (FN)	1000	667	878	858

And the metric table is also extended:

Metric	Custom (TH=0.5)	Custom (TH=0.23)	SKL MLP	Grid Optimum
Accuracy	88.58%	85.16%	89.35%	89.13%
Recall	5.48%	36.48%	17.49%	18.90%
Precision	63.74%	36.20%	67.03%	61.54%
F1-Score	10.09%	36.34%	27.74%	28.92%

A true comparison is best made between the SKL and Grid Search optimised versions as these are based on the same underlying code. There is some improvement in the recall, but not significantly. As expected, there is a trade-off between recall and precision.

Genetic Algorithm Optimisation

I implemented a genetic algorithm version of my 'from scratch' MLP class code. In a standard MLP, back propagation using gradient descent and partial derivatives is used to minimise loss.

In the Genetic Algorithm (GA) version the back propagation process was removed and replaced with evolutionary operators. Rather than calculating gradients, the GA treats the neural network's weights as a chromosome and uses the principles of natural selection to find a solution.

In my existing Python Class, and additional functions were written to run the evolutionary process.

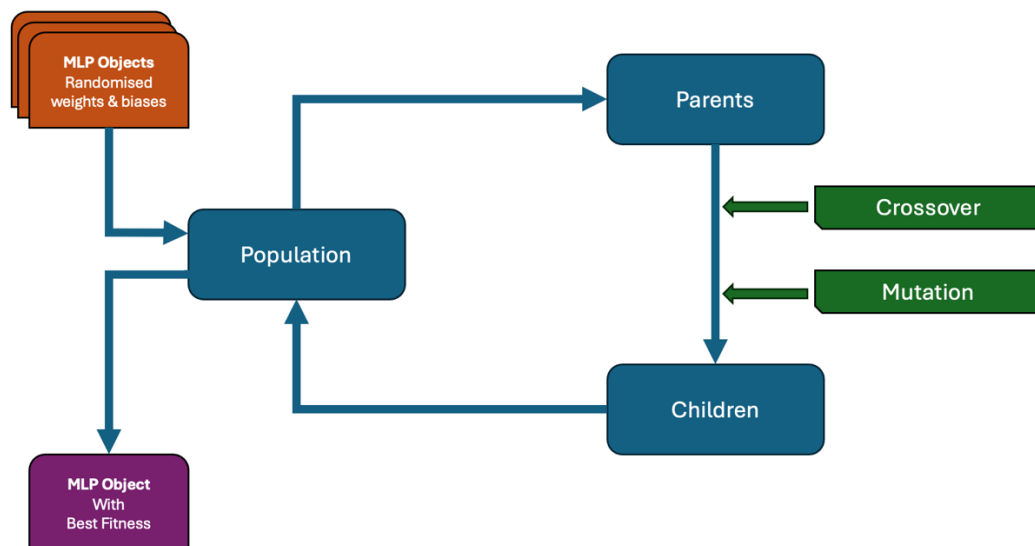


Figure 19 Flow diagram for Genetic Algorithm

The following steps are followed to evolve the 'best' MLP:

- Creation of an initial population of candidate MLP objects.
- Loop through the generations.

For each generation:

1. Calculate the fitness metric for individuals.
2. Sort the individuals based on fitness (in descending order, highest at the top). The fitness function is simply one of the four metrics: accuracy, recall, precision or F1-Score. (Recall is used as the default as this gives the best possible outcome for true positives, but this is tuneable as needed.)
3. Keep the very best individuals each time. This process is known as elitism, where the best survives regardless of their age in the population. This prevents the loss of high-performing traits.
4. Cull the bottom half of the population. The culling rate variable removes the bottom percentage of the population with each generation. This is 50% by default, leaving the top 50% of individuals available to parent the next generation.
5. Generate new children to fill the population using crossover and possible mutation.

Crossover

Pairs of parents exchange 'genetic material' (their weights and biases data), combining successful patterns to create offspring. I chose to find a random point and then swap the 'tails' of data after that point.

Mutation

Random variations are introduced to the chromosome to mimic real life mutations, and to prevent the algorithm from getting stuck in a local minima. A mutation is only applied when a random dice roll is below the mutation rate value (this is user configurable).

To implement the chromosome, the weights and biases are flattened into a single one-dimensional array. To put this back together into arrays after crossover and mutation, I used the stored lengths of the weights and biases to chop up the chromosome.

The first run of the algorithm used the following parameters. Recall was used as the fitness function to give it the best chance of finding the True Positive responses.

Parameter	Value
Fitness Function	Recall
Population Size	20
Generations	20
Culling Rate (Take away bottom N%)	50%
Elite Size (Retain top N)	5
Mutation Rate	0.05 (5%)

The confusion matrix for this run follows:

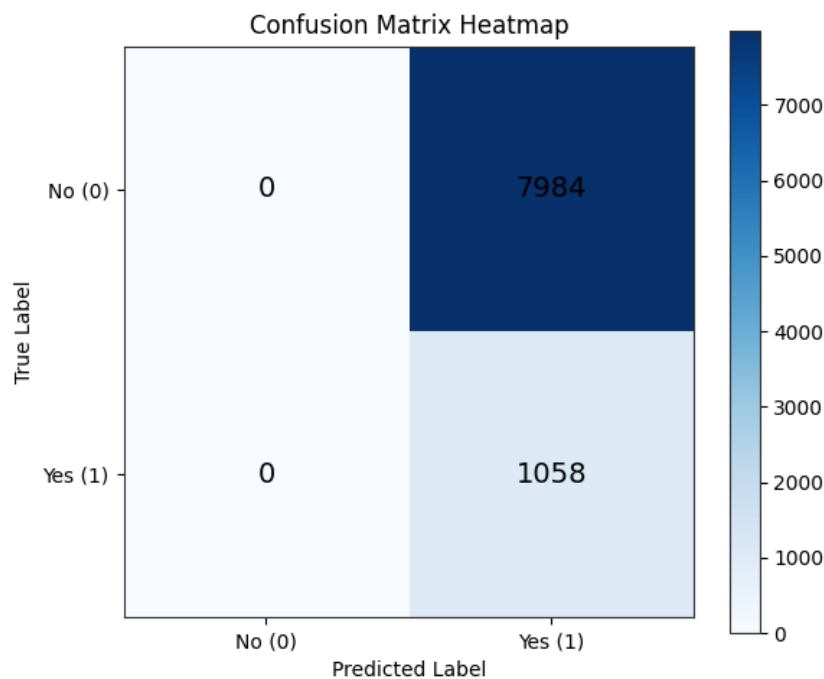


Figure 20 Showing first run of GA, using recall as fitness function

This is bad, but is the best score for Recall that the MLP has generated (99%). Because the fitness function told the GA to maximize recall, it learned that the way to never miss a 'yes' is to predict it for everyone. This is almost the opposite of the original 'from scratch' MLP which tends to predict a negative result.

A better metric to use would be the F1-Score because this is made up of both recall and precision.

Running the same parameters, but specifying the F1-Score metric as the fitness function gives the following confusion matrix:

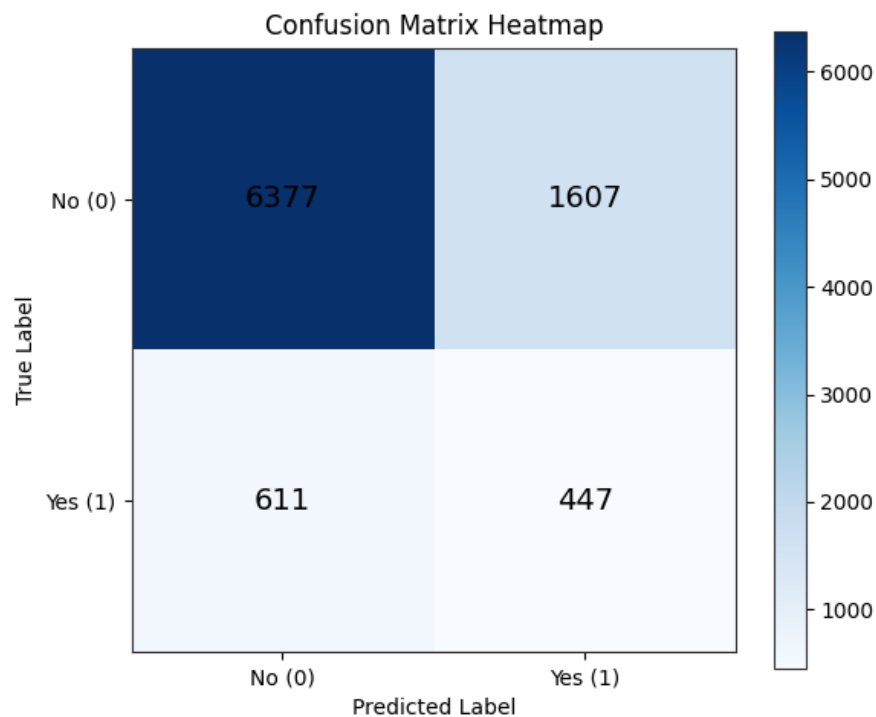


Figure 21 F1-Score GA Based MLP, 20 Generations

Metric	Score	%
Accuracy	0.7547	75.5%
Precision	0.2176	21.8%
Recall	0.4225	42.3%
F1 Score	0.2873	28.7%

The evolution loop records the number of mutations and the best fitness score for each generation. Once this has been plotted, it shows whether the volatility of the mutations results in great leaps in fitness.

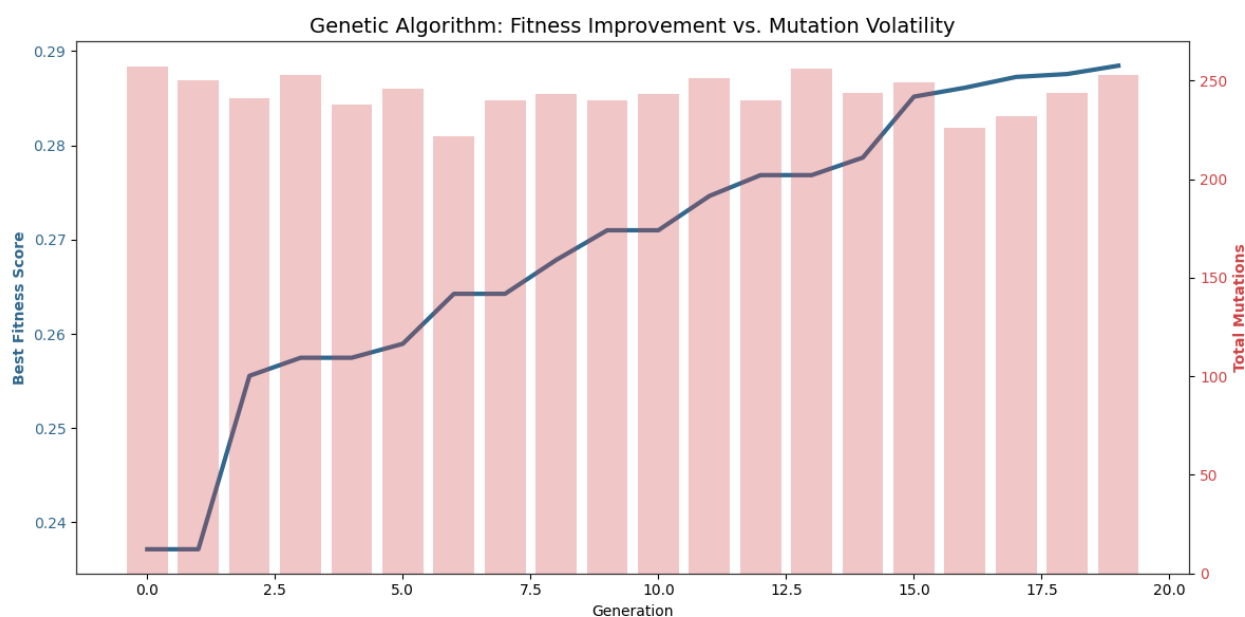


Figure 22 GA MLP, 20 generations, 0.05 Mutation Rate

I also ran the same parameters but with 200 generations and found that only small improvements were made. However, it must be remembered that the initial weights for the individual MLPs are entirely random, as are the mutations and crossovers. It is not possible to have two runs that are identical, and a subsequent run with the same parameters might give better (or worse) results.

Going back to the principles discussed by McCulloch-Pitts (1943), the genetic algorithm applies a wider search space when looking for the optimum way wire together the digital neurons in the network. The randomness in the algorithm means that it potentially tries out wildly different connection configurations rather than changing the existing one in small ways. In the context of the bank dataset, this is highly effective.

A MLP optimised using F1 Score ensures that the model is aggressive enough to identify every possible 'yes' even if it increases the volume of total calls. As discussed before, these extra calls are cheap compared with the loss of value to the bank from missed deposits.

[5234 words]

References

Part 1 – Symbolic AI

1. **Boutell, C. (1893)** *Arms and Armour – In Antiquity and the Middle Ages*, London: Reeves and Turner, Available at: <https://archive.org/details/armsarmourinanti00laco> (Accessed: 16 Feb 2026)
2. **Campbell, N.A. and Reece, J.B. (2005)** *Biology*. 7th edn. Harlow: Pearson. Available at: <https://cssplatformbytha.com/wp-content/uploads/2024/10/Biology-by-Neil-A.-Campbell-Jane-B.-Reece-z-lib.org.pdf> (Accessed: 14 April 2026)
3. **Clark, K.L. (1978)** 'Negation as Failure', in Gallaire, H. and Minker, J. (eds.) *Logic and Data Bases*. New York: Plenum Press, pp. 293–322. Available at: <https://www.doc.ic.ac.uk/~klc/NegAsFailure.pdf> (Accessed :14 April 2026)
4. **McDonald, J. (2015)** *Medieval Weapons & Armour*, Castles and Manor Houses Inc. Available at: <https://www.medievalwarfare.info/weapons.htm> (Accessed: 16 Feb 2026)
5. **Metropolitan Museum of Art (1991)** *Arms and Armor: Permanent Collection*. Available at: https://resources.metmuseum.org/resources/metpublications/pdf/Arms_and_Armor_Permanent_Collection_The_Metropolitan_Museum_of_Art_Bulletin_v_49_no_1_Summer_1991.pdf (Accessed: 16 Feb 2026).
6. **Russell, S.J. and Norvig, P. (2022)** *Artificial Intelligence: A Modern Approach*. 4th edn. Harlow: Pearson. Available at: http://lib.ysu.am/disciplines_bk/efdd4d1d4c2087fe1cbe03d9ced67f34.pdf (Accessed 14 April 2026)
7. **Study.com (n.d.)** *Middle Ages Events | History & Timeline*. Available at: <https://study.com/academy/lesson/major-events-in-the-middle-ages.html> (Accessed: 18 Feb 2026).
8. **Wikipedia (2026)** *Fall of the Western Roman Empire*. Available at: https://en.wikipedia.org/wiki/Fall_of_the_Western_Roman_Empire (Accessed: 18 Feb 2026).
9. **Wikipedia (2026)** *Reformation*. Available at: <https://en.wikipedia.org/wiki/Reformation> (Accessed: 18 Feb 2026).

Part 2 – Connectionist Methods

1. **Bishop, C.M. (2006)** *Pattern Recognition and Machine Learning*. New York: Springer.
2. **Cognism (2024)** *Cold calling success rates: everything you need to know*. Available at: <https://www.cognism.com/blog/cold-calling-success-rates> (Accessed: 18 April 2026)
3. **Deisenroth, M., Faisal, A. A., Ong C. S. (2021)** *Mathematics for Machine Learning*. Cambridge: Cambridge University Press
4. **GeeksforGeeks (2023)** *Adam Optimizer – Deep Learning*. Available at: <https://www.geeksforgeeks.org/deep-learning/adam-optimizer/> (Accessed: 02 May 2026).
5. **Keras (n.d.)** *About Keras*. Available at: https://keras.io/getting_started/about/ (Accessed: 02 May 2026)
6. **McCulloch, W.S. and Pitts, W. (1943)** 'A logical calculus of the ideas immanent in nervous activity', *The Bulletin of Mathematical Biophysics*, 5(4), pp. 115–133.
7. **Moro, S., Rita, P., & Cortez, P. (2014)**. Bank Marketing [Dataset]. UCI Machine Learning Repository. Available at: <https://doi.org/10.24432/C5K306>.
8. **Smith, G. (1998)** *Introductory Mathematics: Algebra and Analysis*. London: Springer-Verlag